VOLUME 05 OF 10

Graphics and Game-Engine Construction

Upper undergraduate

Build a renderer and a small engine instead of only learning an existing editor.

5 modules / 20 lessons / 60 quiz questions

Prepared September 28, 2026

A guided self-study program with original lesson primers, assigned study, engineering labs, checkpoint projects and research work. Not an accredited degree or a substitute for the required practice and feedback.

Volume map

Module 21 From geometry to a software renderer

21.1 Images, color and coordinate conventions • 21.2 Projection and clipping before perspective division • 21.3 Triangle coverage, depth and perspective-correct interpolation • 21.4 Rays, intersections and acceleration structures

Module 22 Explicit GPU programming and graphics APIs

22.1 GPU work submission, resources and pipeline state • 22.2 Buffers, shaders, layouts and resource binding • 22.3 Synchronization, hazards and frames in flight • 22.4 Presentation, resizing and device capability policy

Module 23 Lighting, materials and real-time image quality

23.1 Radiometry, reflectance and physically based materials • 23.2 Textures, mipmaps, filtering and tangent-space detail • 23.3 Shadows, light selection and rendering-path tradeoffs • 23.4 HDR, tone mapping, temporal reconstruction and validation

Module 24 Engine architecture, simulation and resource ownership

24.1 Fixed updates, interpolation and overload policy • 24.2 Entities, components, handles and data-oriented storage • 24.3 Allocators, task graphs and frame-resource lifetime • 24.4 Subsystem boundaries, save formats and reloadability

Module 25 Asset pipelines, editors and an integrated engine

25.1 Importing, cooking and content-addressed asset builds • 25.2 Streaming, residency and memory budgets • 25.3 Editors, inspection and undoable commands • 25.4 Engine integration and reproducible demonstration

Before beginning

Use PROGRAM_GUIDE for sequencing, workload, assessment and the checkpoint project rubric. Use START_HERE for the first session. Reference IDs such as R07 link to assigned resources. The complete reference directory is in PROGRAM_GUIDE and the browser edition.

Answer guidance is separate in ANSWER_KEY.pdf. In the browser edition, answers remain closed until you reveal them. Finish the required study and implementation work rather than treating quiz recall as mastery.

MODULE 21

From geometry to a software renderer

Prerequisites Modules 07-08, 11-16; complete the mathematics checkpoint.

Four lessons

21.1 Images, color and coordinate conventions

Define the numerical and geometric meaning of an image buffer.

21.2 Projection and clipping before perspective division

Transform geometry without confusing visibility with division by depth.

21.3 Triangle coverage, depth and perspective-correct interpolation

Produce crack-resistant filled triangles with correct varying attributes.

21.4 Rays, intersections and acceleration structures

Build a second visibility method and compare its costs with rasterization.

Module project

Build a CPU renderer that writes a portable image file. It must clip geometry, rasterize triangles, resolve visibility and interpolate attributes correctly. Keep it as a reference implementation for future GPU work.

Core references

R19 Carnegie Mellon University — 15-462/662 Computer Graphics

R23 Matt Pharr, Wenzel Jakob, Greg Humphreys — Physically Based Rendering, fourth edition

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 21.1 • MODULE 21

Images, color and coordinate conventions

Learning goal Define the numerical and geometric meaning of an image buffer.

An image buffer stores samples on a two-dimensional grid. Its width, height, channel order, stride and origin must be explicit. A row can include padding, so assuming that every row occupies exactly width times bytes-per-pixel can be wrong. A coordinate convention is a contract shared by geometry, cameras, shaders and tools. Record handedness, axis directions, angle units and matrix multiplication order.

Color values also have meaning. Many image assets use a nonlinear transfer function such as sRGB, while lighting and blending generally need appropriate linear-light values. A normalized channel value is not automatically a physical light measurement. Distinguish color data from non-color textures such as normals or roughness. Treating every texture with the same conversion creates subtle rendering errors.

WORKED EXAMPLE

A 320 by 180 image with four 8-bit channels needs 230,400 bytes of tightly packed pixel storage. A normal-map channel is data; decoding it with a color transfer function changes the represented direction.

PRACTICE + PASS CRITERION

Write a PPM image containing a gradient and coordinate markers. Pass: the dimensions, orientation and channel values are verified by tests, and a README states all coordinate and color assumptions.

ASSIGNED STUDY

R19: image representation and transformations; R23: Radiometry, Spectra, and Color.

CHECK YOUR UNDERSTANDING

1. Why must image stride be recorded?

2. How many bytes are in the tightly packed example image?

3. Should a normal map automatically receive sRGB decoding?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 21.2 • MODULE 21

Projection and clipping before perspective division

Learning goal Transform geometry without confusing visibility with division by depth.

A vertex moves through object, world, view and clip spaces according to the chosen convention. Homogeneous clip coordinates retain information needed to clip primitives before perspective division. Dividing by w maps into normalized device coordinates, but blindly dividing vertices behind or across the eye plane can produce enormous or inverted triangles.

Clipping computes intersections with the view volume and retains the visible portion of a primitive. After clipping, perform perspective division and viewport mapping. Different graphics APIs use different conventions for depth ranges and related details; do not copy a projection matrix without checking its contract. Test points on planes, beyond planes and near degeneracies. Back-face culling is a separate decision based on winding and orientation.

WORKED EXAMPLE

For a clip-space component x=2 and w=4, x/w=0.5. A triangle with vertices on both sides of a clipping plane must be clipped as a primitive; dropping one outside vertex alone destroys its shape.

PRACTICE + PASS CRITERION

Implement clipping against one plane, then the complete chosen clip volume. Pass: triangles crossing the near plane remain finite and correctly shaped, and tests cover points exactly on the boundary.

ASSIGNED STUDY

R19: perspective transformations, clipping and the graphics pipeline; R23: Cameras.

CHECK YOUR UNDERSTANDING

1. Why clip before perspective division?

2. What normalized component follows from x=2 and w=4?

3. Is deleting every outside vertex a valid triangle-clipping algorithm?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 21.3 • MODULE 21

Triangle coverage, depth and perspective-correct interpolation

Learning goal Produce crack-resistant filled triangles with correct varying attributes.

Edge functions determine on which side of a triangle edge a sample lies. Their normalized values yield barycentric coordinates, which sum to one for a nondegenerate triangle. A consistent shared-edge rule prevents neighboring triangles from both missing or inconsistently owning a boundary sample. Degenerate triangles need an explicit rejection policy.

A depth buffer resolves visibility according to the chosen depth representation and comparison rule. Screen-space barycentric interpolation of a general surface attribute is not perspective correct. Interpolate attribute divided by clip w and reciprocal w, then divide the interpolated quantities. Depth interpolation has its own pipeline convention; do not substitute an arbitrary world-space distance. Anti-aliasing estimates pixel coverage rather than pretending a pixel is an infinitesimal point.

WORKED EXAMPLE

For screen-space weights lambda_i, an attribute is a=(sum lambda_i*a_i/w_i)/(sum lambda_i/w_i). When all w values are equal, this reduces to ordinary barycentric interpolation.

PRACTICE + PASS CRITERION

Render two triangles sharing an edge and a receding checkerboard. Pass: no cracks under the documented fill rule, correct overlap ordering, and a visible comparison between affine and perspective-correct texture coordinates.

ASSIGNED STUDY

R19: rasterization, barycentric coordinates and texture mapping; R23: Sampling and Reconstruction.

CHECK YOUR UNDERSTANDING

1. What is the sum of barycentric coordinates?

2. When does perspective-correct interpolation reduce to affine interpolation?

3. Why specify a shared-edge coverage rule?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 21.4 • MODULE 21

Rays, intersections and acceleration structures

Learning goal Build a second visibility method and compare its costs with rasterization.

A ray has origin o and direction d, with points r(t)=o+t*d over a stated interval. The nearest valid positive intersection usually determines a primary visible surface. Directions need not be normalized mathematically, but t is a distance only under suitable normalization and units. Robust intersection tests handle parallel directions, boundary cases and floating-point error.

A bounding-volume hierarchy groups primitives under conservative bounds. Traversal rejects entire groups whose bounds the ray misses. Construction cost, traversal cost, memory and update behavior all matter. An acceleration structure is not a promise of logarithmic worst-case work; difficult overlap can degrade it. Use the brute-force renderer as a correctness oracle before optimizing traversal.

WORKED EXAMPLE

If o=(0,0,0), d=(0,0,2) and t=3, the ray point is (0,0,6); t is not the traveled distance because d has length 2.

PRACTICE + PASS CRITERION

Implement sphere or triangle intersections and a small BVH. Pass: accelerated and brute-force results agree within a defined tolerance over randomized rays, including misses and grazing cases; report build and query costs separately.

ASSIGNED STUDY

R23: Shapes and Primitives and Intersection Acceleration.

CHECK YOUR UNDERSTANDING

1. When is ray parameter t directly a distance?

2. What point occurs in the worked example at t=3?

3. Why retain the brute-force implementation?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 22

Explicit GPU programming and graphics APIs

Prerequisites Module 21 plus Modules 18 and 20. Choose one compatible backend, not all at once.

Four lessons

22.1 GPU work submission, resources and pipeline state

Separate CPU recording from asynchronous GPU execution.

22.2 Buffers, shaders, layouts and resource binding

Match data representations across CPU and GPU boundaries.

22.3 Synchronization, hazards and frames in flight

Express dependencies without serializing the entire renderer.

22.4 Presentation, resizing and device capability policy

Treat the window system as a changing environment rather than a fixed image.

Module project

Implement a GPU backend for the reference scene. Query device capabilities, enable available validation, document the toolchain and handle resize and shutdown safely. A CPU backend remains a valid fallback while hardware access is arranged.

Core references

R21 Khronos Group — Vulkan tutorial

R22 SDL project — SDL3 documentation

R24 Microsoft — Direct3D 12 graphics

R25 Apple — Metal developer resources

R37 NVIDIA — CUDA Programming Guide

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 22.1 • MODULE 22

GPU work submission, resources and pipeline state

Learning goal Separate CPU recording from asynchronous GPU execution.

A graphics API lets software describe resources, pipeline state and work for a GPU. The CPU records or encodes commands, then submits them to queues. Submission does not imply immediate completion. The GPU can still be reading data after the CPU has moved to the next frame. This separation is the central source of lifetime and synchronization mistakes.

A pipeline combines programmable stages with fixed-function configuration and resource interfaces. Vulkan, Direct3D 12 and Metal expose related concepts through different contracts; learning one does not make their details interchangeable. Use the API documentation for the installed version and the selected device. Current tutorial feature requirements must be checked, especially on compatibility layers. Start with a minimal scene before adopting a large abstraction framework.

WORKED EXAMPLE

Recording a draw that references a vertex buffer does not copy the entire buffer into the command stream. Destroying that buffer immediately after submission can invalidate in-flight work.

PRACTICE + PASS CRITERION

Create a window or offscreen target and report supported capabilities. Pass: render a clear color, capture validation output and explain which objects belong to CPU recording versus GPU execution.

ASSIGNED STUDY

R21: Introduction and Drawing a Triangle; alternatively R24 or R25 for one native platform; R22: SDL3 platform setup.

CHECK YOUR UNDERSTANDING

1. Does queue submission mean the GPU has finished?

2. Why can immediate buffer destruction be unsafe?

3. Must a beginner implement three graphics APIs simultaneously?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 22.2 • MODULE 22

Buffers, shaders, layouts and resource binding

Learning goal Match data representations across CPU and GPU boundaries.

Buffers contain bytes; shaders interpret those bytes according to a declared layout. Alignment, padding, matrix convention and element stride must agree between host code and shader code. A C++ structure that looks similar to a shader structure is not proof of binary compatibility. Use explicit offsets or verified layout rules and test small diagnostic values.

Resource binding tells a shader which buffers, textures and samplers it can access. Binding indices and lifetime form an interface contract. Vertex and fragment stages execute with different inputs and responsibilities. Compute shaders are a more general work-dispatch path but still need defined indexing and bounds. Out-of-range threads often occur when dispatch dimensions are rounded upward; guard them rather than assuming exact divisibility.

WORKED EXAMPLE

A workload of 1,000 elements dispatched in groups of 64 needs 16 groups, creating 1,024 logical invocations. The last 24 must not access elements beyond index 999.

PRACTICE + PASS CRITERION

Upload a transform and vertex colors, then add a bounded compute operation. Pass: host/shader layouts are documented, a readback test checks results, and non-multiple work sizes remain correct.

ASSIGNED STUDY

R21: vertex buffers, uniform buffers and compute; R37: execution configuration and memory organization, only for the optional CUDA route.

CHECK YOUR UNDERSTANDING

1. Why can matching field names still produce a layout bug?

2. How many 64-thread groups cover 1,000 elements?

3. What does a resource binding establish?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 22.3 • MODULE 22

Synchronization, hazards and frames in flight

Learning goal Express dependencies without serializing the entire renderer.

A hazard occurs when operations access the same resource in an incompatible order, such as reading before a prior write becomes available and visible. Execution ordering and memory visibility are related but distinct requirements. Barriers, events, semaphores or fences serve different roles depending on the API. Learn their exact scope rather than treating every synchronization primitive as a universal wait.

Frames in flight improve overlap but increase the number of live resources and potentially latency. Per-frame allocations cannot be recycled until the relevant work is complete. A CPU-wide wait after every command may hide a bug while destroying concurrency. First write down the producer, consumer, resource state and required ordering; then choose the narrowest correct mechanism.

WORKED EXAMPLE

A compute pass writes positions and a later draw reads them as vertex data. Correctness requires the appropriate dependency and visibility for that resource, not merely placing source-code statements next to each other.

PRACTICE + PASS CRITERION

Implement a compute-to-draw dependency and two reusable frame contexts. Pass: validation reports no relevant hazards and a stress test never resets a context still used by the GPU.

ASSIGNED STUDY

R21: synchronization, frames in flight and resource transitions; use R24 or R25 equivalents for the selected backend.

CHECK YOUR UNDERSTANDING

1. Are execution ordering and memory visibility identical?

2. When may a frame context be reused?

3. Why is waiting for the whole GPU after each command a poor final design?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 22.4 • MODULE 22

Presentation, resizing and device capability policy

Learning goal Treat the window system as a changing environment rather than a fixed image.

Presentation connects rendered images to a display surface. The set of presentation images, formats and modes is constrained by the platform and device. A window can resize, become minimized or lose its surface. Rebuilding size-dependent resources requires coordination with in-flight work. A zero-sized drawable should not trigger invalid allocations or divide-by-zero camera calculations.

Define a feature policy: required capabilities, optional enhancements and a fallback. Record why a device is unsupported instead of failing with an obscure assertion. Device errors must preserve useful diagnostics; recovery strategy can range from a clear restart request to a tested rebuild. Keep platform services behind a small interface, while acknowledging that platform-specific behavior cannot always be abstracted away.

WORKED EXAMPLE

A minimized window can report a drawable size of zero. The application should suspend or skip size-dependent rendering until a usable size returns, rather than allocating a zero-height depth target.

PRACTICE + PASS CRITERION

Stress resize, minimize, restore and orderly shutdown. Pass: no stale size-dependent resources, no division by zero, and a readable capability report explains enabled features and fallbacks.

ASSIGNED STUDY

R21: swapchain recreation and device selection; R22: window events; R24 or R25: presentation and resource lifetime.

CHECK YOUR UNDERSTANDING

1. Why can resizing require more than changing the viewport?

2. What should happen for a zero-height drawable?

3. What is the purpose of an explicit feature policy?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 23

Lighting, materials and real-time image quality

Prerequisites Modules 07, 09-10, 21-22.

Four lessons

23.1 Radiometry, reflectance and physically based materials

Explain how lighting models approximate light transport.

23.2 Textures, mipmaps, filtering and tangent-space detail

Control aliasing while preserving the meaning of texture data.

23.3 Shadows, light selection and rendering-path tradeoffs

Choose a lighting architecture from scene and bandwidth requirements.

23.4 HDR, tone mapping, temporal reconstruction and validation

Separate display mapping from reconstruction and simulation correctness.

Module project

Create a material-and-lighting demonstration with a documented color pipeline. Compare image quality and frame cost at identical resolution and scene content; include reference images, not only attractive screenshots.

Core references

R19 Carnegie Mellon University — 15-462/662 Computer Graphics

R23 Matt Pharr, Wenzel Jakob, Greg Humphreys — Physically Based Rendering, fourth edition

R21 Khronos Group — Vulkan tutorial

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 23.1 • MODULE 23

Radiometry, reflectance and physically based materials

Learning goal Explain how lighting models approximate light transport.

Radiometry gives precise meanings to quantities such as radiant flux, irradiance and radiance. A bidirectional reflectance distribution function describes how incoming light contributes to outgoing light at a surface. Physically motivated models respect appropriate energy and reciprocity constraints, although real-time implementations often introduce approximations.

A material parameter is meaningful only within its model and encoding. Roughness controls a distribution of microscopic surface orientations in common microfacet models; it is not simply a blur slider. Metallic and dielectric materials have different reflectance behavior. Normals, tangent frames and units must agree. A plausible image can still be numerically or physically inconsistent, so test simple controlled scenes before judging a complex one.

WORKED EXAMPLE

For an ideal diffuse surface, the Lambertian BRDF is reflectance divided by pi. Increasing reflectance beyond one would violate the intended passive energy-conservation interpretation.

PRACTICE + PASS CRITERION

Implement diffuse lighting and one documented microfacet model. Pass: test a constant environment, grazing angles and roughness limits, and state approximations rather than labeling any shiny shader physically exact.

ASSIGNED STUDY

R23: Radiometry, Reflection Models and Light Transport; R19: shading.

CHECK YOUR UNDERSTANDING

1. What does a BRDF relate?

2. What is the Lambertian BRDF for reflectance rho?

3. Why document roughness encoding?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 23.2 • MODULE 23

Textures, mipmaps, filtering and tangent-space detail

Learning goal Control aliasing while preserving the meaning of texture data.

Textures are sampled signals attached to geometry. When many source texels cover one output pixel, selecting a single texel aliases. Mipmaps prefilter a hierarchy of resolutions; sampling chooses suitable levels and combines samples according to the filtering method. Anisotropic footprints require more than a single isotropic level to preserve oblique detail.

Generate filtered color in an appropriate linear representation, and treat normal or data maps according to their semantics. Tangent-space normal mapping depends on a consistent tangent basis, handedness and mesh UVs. Seams, mirrored UVs and degenerate triangles are valuable tests. Compression trades storage and bandwidth for error; evaluate error in use, not only as a file-size ratio.

WORKED EXAMPLE

A 256 by 256 base texture contains 65,536 texels. A complete power-of-two mip chain adds about one third as many texels beyond the base, approaching four thirds of the base total.

PRACTICE + PASS CRITERION

Render a receding textured floor with and without mipmaps and inspect a normal-mapped seam. Pass: report texture storage including mip levels and explain why non-color data needs its own filtering policy.

ASSIGNED STUDY

R23: Textures and Sampling and Reconstruction; R21: texture images, samplers and mipmaps.

CHECK YOUR UNDERSTANDING

1. What problem do mipmaps primarily address?

2. What is the approximate total mip-chain storage factor?

3. Why can mirrored UVs break normal mapping?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 23.3 • MODULE 23

Shadows, light selection and rendering-path tradeoffs

Learning goal Choose a lighting architecture from scene and bandwidth requirements.

A shadow map stores depth from a light viewpoint, allowing a later surface sample to test visibility. Resolution, bias and filtering introduce tradeoffs: too little bias causes self-shadow artifacts, while too much separates shadows from contact. A ray-traced visibility query has different costs and artifacts, not automatic perfection.

Forward rendering evaluates material shading while drawing geometry. Deferred approaches store intermediate surface information and light it later, trading bandwidth and storage for different lighting behavior. Tiled or clustered light selection limits which lights a region considers. Transparency, multisampling, material diversity and platform bandwidth affect the choice. Count bytes and work rather than selecting a path because a large studio uses it.

WORKED EXAMPLE

A hypothetical 1,920 by 1,080 G-buffer with 16 bytes per pixel occupies about 33.2 MB for one set of attachments, before depth, extra buffers or repeated reads and writes.

PRACTICE + PASS CRITERION

Implement shadow mapping and compare a simple forward path with tiled light selection or a small deferred prototype. Pass: disclose buffer sizes, light counts, transparency limitations and measured frame costs.

ASSIGNED STUDY

R19: real-time rendering and shadows; R23: Lights and Light Transport.

CHECK YOUR UNDERSTANDING

1. Why can excessive shadow bias be visible?

2. Does deferred rendering remove bandwidth cost?

3. What determines whether clustered lighting helps?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 23.4 • MODULE 23

HDR, tone mapping, temporal reconstruction and validation

Learning goal Separate display mapping from reconstruction and simulation correctness.

High-dynamic-range rendering retains scene values beyond a narrow display range. Tone mapping converts those values for a target presentation, while exposure controls the chosen scale. These are not substitutes for correct lighting units or color conversion. Display capabilities and output formats must be handled explicitly.

Temporal techniques reuse earlier samples, often after motion-based reprojection. History can reduce noise or aliasing but can also create ghosting when visibility changes or motion vectors are wrong. Detect invalid history, including disocclusion and camera cuts. A single still screenshot misses temporal artifacts. Evaluate moving sequences, latency, stability and error against a suitable reference. An image-quality metric is evidence, not a complete description of perceptual quality.

WORKED EXAMPLE

A newly revealed background pixel has no valid previous foreground history. Reusing that history without rejection can leave a trailing silhouette even when each individual frame is numerically finite.

PRACTICE + PASS CRITERION

Add exposure, a documented tone mapper and a basic temporal accumulation experiment. Pass: capture a camera cut and disocclusion test, show failure modes, and compare both still-image error and motion behavior.

ASSIGNED STUDY

R23: Film and Imaging and Sampling; R19: anti-aliasing and image processing.

CHECK YOUR UNDERSTANDING

1. Are tone mapping and correct light transport the same task?

2. Why reject history at a camera cut?

3. Why are still screenshots insufficient for temporal methods?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 24

Engine architecture, simulation and resource ownership

Prerequisites Modules 12, 17-18, 21-23.

Four lessons

24.1 Fixed updates, interpolation and overload policy

Separate simulation time from rendering time and real elapsed time.

24.2 Entities, components, handles and data-oriented storage

Design identity separately from memory location.

24.3 Allocators, task graphs and frame-resource lifetime

Make temporary storage and parallel dependencies observable.

24.4 Subsystem boundaries, save formats and reloadability

Choose interfaces that make change and failure manageable.

Module project

Build a small engine with a fixed-step world, generational entity handles, explicit resource ownership and an inspectable task graph. Keep the game rules separate enough to run automated headless tests.

Core references

R20 Carnegie Mellon University — 15-466/666 Computer Game Programming

R26 Epic Games — Programming with C++ in Unreal Engine

R27 Glenn Fiedler — Fix Your Timestep!

R11 C++ Core Guidelines editors — C++ Core Guidelines

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 24.1 • MODULE 24

Fixed updates, interpolation and overload policy

Learning goal Separate simulation time from rendering time and real elapsed time.

A fixed-step simulation advances by a chosen interval, while rendering may occur at a different rate. An accumulator collects elapsed time and runs enough simulation steps to catch up. Rendering can interpolate between stored simulation states using the leftover fraction. This improves presentation smoothness without changing the simulation step itself.

The system needs an overload policy. An unbounded catch-up loop can consume more time than it recovers. Capping work, dropping accumulated time or reducing quality have different consequences and must be documented. Fixed stepping alone does not guarantee deterministic results: input timing, floating-point behavior, iteration order and parallel execution can still differ. Keep wall-clock measurement outside the deterministic state transition where possible.

WORKED EXAMPLE

With a 10 ms simulation step and 25 ms accumulated, two updates consume 20 ms and leave 5 ms. The interpolation fraction is 0.5 between the relevant stored states.

PRACTICE + PASS CRITERION

Implement an accumulator loop and inject long frames. Pass: simulation advances consistently under normal load, interpolation is bounded, and the overload behavior is tested rather than silently spiraling.

ASSIGNED STUDY

R27: Fix Your Timestep!; R20: game loops and timing. Use the article as a starting model, not a guarantee of determinism.

CHECK YOUR UNDERSTANDING

1. What fraction remains after two 10 ms steps from 25 ms?

2. Does a fixed timestep alone guarantee determinism?

3. Why must catch-up work be bounded or otherwise controlled?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 24.2 • MODULE 24

Entities, components, handles and data-oriented storage

Learning goal Design identity separately from memory location.

An entity is an identity to which state can be attached. A component represents a category of data or behavior according to the chosen architecture. An entity-component-system design is useful when it improves iteration, separation and tooling; it is not a universal requirement for every object.

A handle can contain a slot index and a generation counter. Reusing a slot changes its generation, allowing stale handles to be rejected. Raw pointers into moving arrays cannot provide the same safety automatically. Sparse sets, archetype chunks and ordinary arrays have different iteration and mutation costs. Structural changes during iteration require a policy, such as deferred commands. Evaluate representative access patterns rather than maximizing architectural complexity.

WORKED EXAMPLE

Handle (slot 7, generation 3) must not refer to a new entity placed in slot 7 with generation 4. Matching the index alone would mistakenly resurrect a destroyed identity.

PRACTICE + PASS CRITERION

Implement a small generational entity store with two components and deferred destruction. Pass: stale handles fail safely, iteration survives queued changes, and benchmarks include creation, deletion and common queries.

ASSIGNED STUDY

R20: game-state organization; R26: gameplay architecture for comparison; R11: lifetime and interface guidance.

CHECK YOUR UNDERSTANDING

1. Why separate identity from an object address?

2. Is handle (7,3) valid for a slot now at generation 4?

3. Why defer structural changes during a component iteration?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 24.3 • MODULE 24

Allocators, task graphs and frame-resource lifetime

Learning goal Make temporary storage and parallel dependencies observable.

Arena allocation groups objects with a shared lifetime and can make allocation and reset inexpensive. It is safe only when no surviving references point into reset storage. A frame arena may outlive the CPU frame when asynchronous jobs or the GPU still reference it. Allocation strategy follows lifetime, not the other way around.

A task graph declares dependencies among units of work. A frame graph can additionally describe resource reads and writes so ordering, transient allocation and synchronization can be derived. Hidden global state defeats this model. Start with a correct serial execution of the graph, then parallelize independent work. Instrument queue delay, execution duration and critical-path length rather than reporting only total task count.

WORKED EXAMPLE

Tasks A and B can run concurrently, but C reading both outputs waits for both. Adding more workers cannot shorten C or remove that dependency.

PRACTICE + PASS CRITERION

Create a serial-then-parallel task graph and a checked temporary arena. Pass: resetting live storage is detected in tests, dependency order is verified, and the critical path is identified from a trace.

ASSIGNED STUDY

R11: ownership and concurrency; R20: engine architecture; R26: engine systems as a comparative implementation.

CHECK YOUR UNDERSTANDING

1. When is resetting an arena safe?

2. Can task C consume A before A has completed its required output?

3. Why implement a serial graph executor first?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 24.4 • MODULE 24

Subsystem boundaries, save formats and reloadability

Learning goal Choose interfaces that make change and failure manageable.

An engine subsystem should expose a small contract for initialization, updates, resource ownership and shutdown. Avoid a global manager that silently depends on every other manager. Explicit dependency order makes partial initialization failure and test construction easier to reason about. A plugin boundary adds binary and versioning constraints beyond ordinary source-level interfaces.

Save data should represent durable game meaning rather than raw memory addresses or accidental object layout. Version formats, validate input and define migration or rejection behavior. Hot reload is especially difficult when code, layouts or resources change while old references remain. A safe educational implementation can stop the world, unload controlled state and rebuild it rather than pretending arbitrary live code replacement is simple.

WORKED EXAMPLE

A saved entity reference uses a stable serialized identifier and a reconstruction table, not a pointer value from the previous process. A missing referenced asset needs a fallback or explicit error.

PRACTICE + PASS CRITERION

Add versioned saves and one reloadable asset type. Pass: old supported data migrates, malformed data is rejected without partial world corruption, and subsystem shutdown works after injected initialization failure.

ASSIGNED STUDY

R20: resource management and tools; R26: serialization and gameplay architecture; R11: interfaces and error handling.

CHECK YOUR UNDERSTANDING

1. Why is a raw memory pointer unsuitable for a durable save?

2. What must a format version enable?

3. Why test partial initialization failure?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 25

Asset pipelines, editors and an integrated engine

Prerequisites Module 24 and the software-engineering practices of Module 20.

Four lessons

25.1 Importing, cooking and content-addressed asset builds

Treat assets as compiled inputs with dependency tracking.

25.2 Streaming, residency and memory budgets

Load content without allowing asynchronous work to violate ownership.

25.3 Editors, inspection and undoable commands

Make engine state understandable and safely editable.

25.4 Engine integration and reproducible demonstration

Deliver a small complete system whose limitations are measurable.

Module project

Gate E: deliver a small 3D engine demonstration with an asset cooker, reproducible scene, inspector, save/load, diagnostics and a measured frame/memory budget. A second person must build and run it from the README.

Core references

R20 Carnegie Mellon University — 15-466/666 Computer Game Programming

R26 Epic Games — Programming with C++ in Unreal Engine

R22 SDL project — SDL3 documentation

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 25.1 • MODULE 25

Importing, cooking and content-addressed asset builds

Learning goal Treat assets as compiled inputs with dependency tracking.

Authoring assets prioritize editability; runtime assets prioritize loading and execution. An importer reads a source representation, while a cooker transforms it into a platform-appropriate runtime form. The build result depends on source bytes, settings, tool version and dependencies. Caching only by filename can return stale or incorrect output.

A content-addressed key can combine these inputs into a digest. Dependency graphs determine what must be rebuilt after a change. Reproducibility requires stable ordering and controlled metadata, not only hashing the final file. Validate imported geometry, dimensions and resource limits because assets are untrusted inputs. Keep source provenance and license information with the pipeline rather than rediscovering it before release.

WORKED EXAMPLE

Changing a texture compression setting must invalidate the cooked texture even when the original PNG is unchanged. Otherwise the cache key is missing a meaningful input.

PRACTICE + PASS CRITERION

Build a cooker for one mesh and one image representation. Pass: identical declared inputs reproduce the same result, changed settings invalidate the cache, and corrupted sources produce readable failures.

ASSIGNED STUDY

R20: asset pipelines; R26: asset management and content tools. Apply Module 20 dependency and provenance practices.

CHECK YOUR UNDERSTANDING

1. Why distinguish source assets from runtime assets?

2. Must a compression-setting change invalidate a cached build?

3. Why include tool version in a cache key?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 25.2 • MODULE 25

Streaming, residency and memory budgets

Learning goal Load content without allowing asynchronous work to violate ownership.

Streaming moves needed content through storage, CPU memory and device memory under a budget. An asset can be requested, loading, decoded, uploaded, resident or failed. Model these states explicitly. A cancellation request must not leave callbacks writing into destroyed owners. Residency policy chooses what remains available when total demand exceeds the budget.

Peak memory matters: compressed input, decoded staging data and final GPU resources can temporarily coexist. Loading one asset at a time is not always sufficient if each individual asset is too large. Level of detail and fallback resources can preserve responsiveness while content is incomplete. Measure misses, stalls and quality changes under controlled constrained budgets.

WORKED EXAMPLE

A 20 MB compressed asset may expand to 100 MB and require another staging copy. Budgeting only the on-disk size substantially understates the transient memory requirement.

PRACTICE + PASS CRITERION

Implement a bounded streaming queue with placeholder assets and cancellation. Pass: a forced small budget produces graceful fallback, no use-after-free, and a report of peak rather than only steady-state memory.

ASSIGNED STUDY

R20: resource loading; R26: asset management, asynchronous loading and streaming concepts.

CHECK YOUR UNDERSTANDING

1. Why is compressed file size not the complete memory budget?

2. What should rendering do while an optional asset is unavailable?

3. Why model loading as explicit states?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 25.3 • MODULE 25

Editors, inspection and undoable commands

Learning goal Make engine state understandable and safely editable.

An editor is a program for manipulating structured state, not merely a collection of buttons. Separate the document or scene model from its views. Commands can capture reversible changes for undo and redo. Some operations are not trivially reversible, so record enough prior state or use a well-defined transaction.

Inspection tools expose entity identity, transforms, resources, timings and errors. Good tools shorten debugging loops and let non-programmers test assumptions. Input focus matters when an editor and game view share keys. Changes made during a running simulation need a policy: transient preview, committed authoring change or explicit rejection. Keep data validation active in tools; editor-generated data can still be invalid.

WORKED EXAMPLE

A move command can store the entity identifier, old transform and new transform. Undo restores the old transform only if the target identity remains valid under the editor contract.

PRACTICE + PASS CRITERION

Create a scene inspector with transform editing and a ten-command undo/redo test. Pass: invalid inputs are rejected, focus does not trigger unintended gameplay, and save/reload preserves committed changes.

ASSIGNED STUDY

R20: game tools; R26: editor and reflection concepts. Use a minimal UI toolkit appropriate to the selected platform.

CHECK YOUR UNDERSTANDING

1. What is the model/view distinction useful for?

2. What data can make a transform move reversible?

3. Why distinguish preview state from committed authoring state?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 25.4 • MODULE 25

Engine integration and reproducible demonstration

Learning goal Deliver a small complete system whose limitations are measurable.

Integration exposes mismatched assumptions that isolated components hide: units, coordinate systems, thread ownership, asset versions and shutdown order. A useful milestone freezes a small scene and tests the complete path from source assets to presentation. It is better to finish a constrained engine than to accumulate disconnected advanced features.

Define a target machine and operating conditions before claiming a frame budget. Report CPU and GPU timings separately where available, plus resolution, scene counts, build mode and peak memory. Use percentiles or time-series evidence to reveal stutter. A successful demonstration also needs clear controls, error messages and a clean build process. Reviewers should be able to reproduce the result without undocumented files on the author's computer.

WORKED EXAMPLE

A claimed 60 frames per second implies a 16.67 ms frame interval, but an average of 16 ms can still hide repeated 60 ms stalls. The distribution matters to the declared experience.

PRACTICE + PASS CRITERION

Complete Gate E using the project specification in the handbook. Pass: another person follows the README, a headless test scene passes, all assets have provenance, and a limitations section identifies remaining correctness or performance risks.

ASSIGNED STUDY

R20: project completion; R26: profiling and engine architecture comparison; revisit Module 20 release and review practices.

CHECK YOUR UNDERSTANDING

1. What frame interval corresponds to 60 frames per second?

2. Why report more than average frame time?

3. What does a clean-machine build test reveal?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.